

FastAPI Notes

Intermediate Concepts — Request Bodies, Validation & Async

1. Request Bodies

A request body is data sent by the client in the body of an HTTP request (typically with POST, PUT, or PATCH). FastAPI uses **Pydantic models** to declare, parse, and validate request bodies automatically.

Defining a Pydantic Model

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    in_stock: bool = True

@app.post("/items/")
async def create_item(item: Item):
    return {"item_name": item.name, "item_price": item.price}
```

FastAPI reads the type hints on the `Item` class, validates the incoming JSON against them, converts it into an `Item` instance, and exposes it as the `item` parameter inside the function.

Combining Path, Query, and Body Parameters

FastAPI distinguishes parameter sources automatically: path parameters come from the URL path, simple types (`str`, `int`, `bool`, etc.) not in the path are treated as query parameters, and Pydantic model types are treated as the request body.

```
@app.put("/items/{item_id}")
async def update_item(item_id: int, q: str | None, item: Item):
    result = {"item_id": item_id, **item.model_dump()}
    if q:
        result.update({"q": q})
    return result
```

Note: In Pydantic v2 (used by current FastAPI), use `model_dump()` instead of the deprecated `.dict()` method.

2. Validation

Pydantic gives you fine-grained control over validation beyond basic type checks — constraints, custom rules, and clear error messages, all enforced before your endpoint logic ever runs.

Field Constraints

```
from pydantic import BaseModel, Field

class Item(BaseModel):
    name: str = Field(min_length=1, max_length=50)
    price: float = Field(gt=0, description="Price must be greater than zero")
    tax: float | None = Field(default=None, ge=0, le=1)
```

Common constraint keywords: `gt`, `ge`, `lt`, `le` for numbers, and `min_length` / `max_length` for strings.

Custom Validators

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    username: str
    age: int

    @field_validator("username")
    @classmethod
    def no_spaces(cls, v):
        if " " in v:
            raise ValueError("username must not contain spaces")
        return v
```

Validation Errors

When validation fails, FastAPI automatically returns a `422 Unprocessable Entity` response with a JSON body describing exactly which field failed and why — no manual error handling required for standard cases.

Query & Path Parameter Validation

```
from fastapi import FastAPI, Query, Path

app = FastAPI()

@app.get("/items/{item_id}")
async def read_item(
    item_id: int = Path(gt=0),
    q: str | None = Query(default=None, max_length=20),
):
    return {"item_id": item_id, "q": q}
```

Response Validation with `response_model`

```
class ItemOut(BaseModel):
    name: str
    price: float

@app.post("/items/", response_model=ItemOut)
```

```
async def create_item(item: Item):
    return item # extra fields on `item` are stripped from the response
```

`response_model` filters and validates the outgoing data too, ensuring you never leak fields you didn't intend to expose (e.g. a hashed password on a `User` model).

3. Async in FastAPI

FastAPI fully supports both regular `def` and `async def` path operation functions. Choosing correctly matters for performance under load.

async def vs def

- Use `async def` when your function awaits non-blocking I/O (an async database driver, async HTTP client, etc.).
- Use plain `def` when your code is synchronous or blocking (a regular DB driver, CPU-bound work, blocking file I/O) — FastAPI automatically runs these in a separate thread pool so they don't block the event loop.
- Never call blocking code directly inside an `async def` function without awaiting it properly; that blocks the whole event loop and stalls every other concurrent request.

Basic async Endpoint

```
import asyncio

@app.get("/ping")
async def ping():
    await asyncio.sleep(1) # simulates non-blocking I/O
    return {"message": "pong"}
```

Async Database Calls

```
# Example using an async ORM/driver (e.g. SQLAlchemy async session)
@app.get("/users/{user_id}")
async def get_user(user_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == user_id))
    user = result.scalar_one_or_none()
    return user
```

Background Tasks

For work that should run after the response is sent (e.g. sending an email or logging), use `BackgroundTasks` instead of awaiting it inline:

```
from fastapi import BackgroundTasks

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(message + "\n")
```

```
@app.post("/notify/")
async def notify(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(write_log, f"notification sent to {email}")
    return {"message": "notification sent"}
```

Note: Mixing sync and async correctly is the single biggest performance lever in FastAPI apps. A blocking call inside `async def` can silently degrade throughput far more than using plain `def` for that same endpoint would.

Quick Reference

Concept	Key Tool	Typical Use
Request body	Pydantic BaseModel	Parse & validate JSON payloads
Field constraints	Field(gt=, max_length=, ...)	Enforce value rules
Custom rules	@field_validator	Cross-field / custom logic
Response shaping	response_model=	Filter & validate output
Non-blocking I/O	async def + await	DB calls, HTTP requests
Blocking work	def (sync)	CPU-bound / blocking libraries
Post-response work	BackgroundTasks	Emails, logging, cleanup